IT314 Take-Home Assignment 2

AI-Powered Chrome Extension for Real-Time Requirement Analysis

Submitted by: Manya Rana - 202401115

Abstract
Requirements engineering meetings often contain statements that are understandable to people in the discussion but are too vague, incomplete, or difficult to test to serve as high-quality software requirements. This project presents an AI Requirement Assistant implemented as a Chrome Extension with a FastAPI backend and a Gemini-based AI analysis layer. The system monitors live meeting captions, identifies potentially ambiguous requirements, generates clarification questions, accepts stakeholder responses, and converts the clarified information into Functional Requirements (FRs) or Non-Functional Requirements (NFRs). It also evaluates the quality of requirements before and after clarification and maintains an exportable requirements history.

The demonstration scenario is an AI-based resume analyzer. Example vague statements such as "The system should be fast" can be converted, after clarification, into measurable requirements such as "The system shall process each resume within 3 seconds." The system therefore demonstrates how AI can support requirements elicitation, clarification, classification, and quality improvement during a live meeting.

1. Introduction
Software requirements are the foundation of software development. If requirements are vague or incomplete, later activities such as design, implementation, testing, and validation can be affected. In a live stakeholder meeting, participants may use natural language such as "the system should be fast," "the model should be accurate," or "the ranking should be fair." These statements communicate intent, but they do not always provide enough measurable information for implementation and testing.

The proposed AI Requirement Assistant acts as a requirements-engineering assistant during a live meeting. Instead of treating every statement as a finished requirement, it attempts to identify genuine ambiguity and asks a focused clarification question. The stakeholder's answer is then used to create a more precise requirement.

2. Problem Statement
Stakeholders frequently express software needs in incomplete or ambiguous natural language. Important constraints such as performance thresholds, evaluation criteria, fairness expectations, explainability requirements, or delivery deadlines may be omitted. If these ambiguities are not resolved during requirements elicitation, the resulting requirements may be difficult to implement consistently or verify objectively.

The project therefore addresses the following problem: how can an AI-assisted tool monitor requirements discussed during a live meeting, detect statements that need clarification, ask useful questions, incorporate stakeholder answers, and demonstrate that the resulting requirements are of higher quality?

3. Objectives
- Capture or monitor live meeting caption text through a Chrome Extension.
- Identify requirement statements that are vague, incomplete, ambiguous, or difficult to test.
- Generate one focused clarification question when clarification is materially useful.
- Accept the stakeholder's clarification response.
- Generate a clear and testable refined requirement.
- Classify the refined requirement as a Functional Requirement (FR) or Non-Functional Requirement (NFR).

- Categorize NFRs using categories such as Performance, Security, Reliability, Fairness, Usability, Explainability, Scalability, or Other.
- Evaluate requirement quality before and after clarification using Clarity, Completeness, Testability, and Ambiguity.
- Maintain a requirements history and provide an export of the collected requirements and evaluation data.

4. Scope

The scope of the prototype includes:

- Live caption monitoring in a supported browser-based meeting environment.
- AI-assisted ambiguity analysis.
- Clarification-question generation.
- Stakeholder response capture.
- FR/NFR generation and classification.
- Before-versus-after quality evaluation.
- Requirements history and JSON export.

The prototype is intended as an academic demonstration of AI-assisted requirements engineering. It does not attempt to replace a professional requirements engineer, provide a complete enterprise requirements management system, or implement a production-grade integration with every meeting platform.

5. Demonstration Scenario

The assignment scenario is an AI-based resume analyzer. The stakeholder wants the system to automatically shortlist candidates and rank them according to their relevance to a job description. The discussion contains several statements that are useful for demonstrating requirements clarification.

Stakeholder statement	Potential issue	Clarification direction
Rank candidates based on relevance.	Meaning of relevance is unspecified.	Ask which factors determine relevance.
Consider overall profile strength.	Profile strength is not defined.	Ask what characteristics indicate profile strength.
Prioritize years of experience, but not strictly.	Relative weighting is unclear.	Ask how experience should influence ranking.
It should be good enough so HR trusts it.	Accuracy/trust threshold is not measurable.	Ask for an acceptable evaluation threshold.
Explainability would be useful.	Expected explanation content is unspecified.	Ask what information the explanation should contain.
It shouldn't be slow.	Performance target is not measurable.	Ask for a maximum processing time.
We need an MVP soon.	Delivery deadline is unspecified.	Ask for a target date or time window.

6. Requirements Engineering Concepts Applied

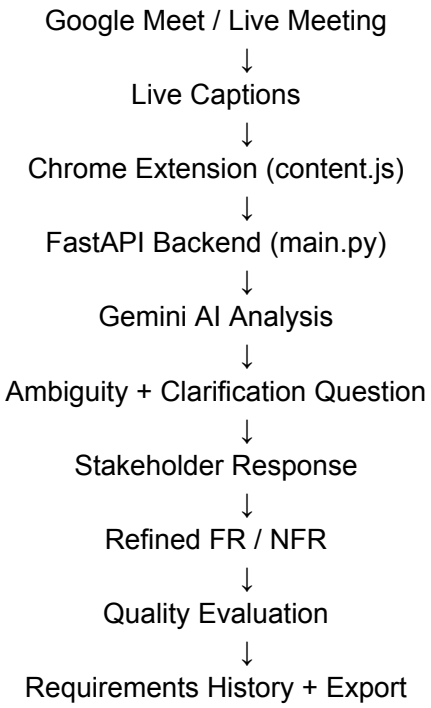
6.1 Functional Requirements

A Functional Requirement specifies what the system should do. In this project, examples include parsing resumes, ranking candidates, comparing resumes with job descriptions, and providing explanations for rankings.

6.2 Non-Functional Requirements
A Non-Functional Requirement describes a quality attribute, constraint, or condition under which the system should operate. Examples include performance, fairness, reliability, security, usability, and explainability.

6.3 Ambiguity and Testability
The prototype treats vague or non-measurable wording as a possible reason for clarification. The goal is not to force unnecessary implementation details, but to ask a question when the missing information would materially improve the requirement.

7. System Architecture
The system follows the architecture below:



7.1 Chrome Extension
The Chrome Extension contains the meeting-page content script. It monitors visible meeting caption text, waits for a stable caption, and sends a detected requirement to the backend. The extension also provides the user interface for analysis results, clarification responses, refined requirements, quality evaluation, history, and export.

7.2 FastAPI Backend
The backend provides REST endpoints for the three main AI-assisted operations: analyzing a requirement, generating a refined requirement, and evaluating requirement quality. Cross-Origin Resource Sharing (CORS) is enabled so the browser extension can communicate with the local backend.

7.3 Gemini AI Layer
Gemini is used to interpret natural-language stakeholder statements. Structured JSON output is requested so that the frontend can display ambiguity information, clarification questions, refined requirements, classifications, and quality scores.

8. Implementation

8.1 Caption Monitoring

The content script reads the visible meeting page text and extracts the speaker's current caption. Because live captions can change incrementally as a sentence is spoken, the prototype waits for the caption to remain stable before sending it for analysis. A last-analyzed value is also maintained to reduce repeated requests for the same text.

8.2 Requirement Analysis Endpoint

POST /analyze-requirement accepts a statement and asks the AI whether it is genuinely ambiguous, vague, incomplete, or difficult to test. If clarification is required, the response contains an explanation and one focused clarification question.

8.3 Requirement Generation Endpoint

POST /generate-requirement accepts the original statement, clarification question, and stakeholder response. The AI then generates one clear and testable requirement, classifying it as FR or NFR and providing an NFR category where appropriate.

8.4 Requirement Evaluation Endpoint

POST /evaluate-requirement compares the original and refined requirements. Four dimensions are scored from 1 to 5: Clarity, Completeness, Testability, and Ambiguity. Higher scores indicate better quality, including a higher score for Ambiguity meaning less ambiguity.

8.5 Requirements History

The extension maintains a history of refined requirements. Each entry stores the original statement, ambiguity, clarification question, stakeholder response, refined requirement, FR/NFR type, category, and evaluation information.

8.6 Export

The requirements history can be exported as a JSON file. The export contains project information, a summary count of functional and non-functional requirements, and the detailed requirement records.

9. Technologies Used

Technology	Purpose
HTML/CSS/JavaScript	Chrome Extension user interface and content-script logic.
Google Chrome Extension Manifest V3	Extension configuration and content-script registration.
Python	Backend implementation language.
FastAPI	REST API framework for communication between the extension and AI layer.
Pydantic	Request-data validation for backend endpoints.
python-dotenv	Loading the Gemini API key from the local environment.
Google GenAI SDK	Communication with the Gemini model.
Gemini	Natural-language requirements analysis, refinement, and evaluation.
JSON	Structured AI responses and requirements export.

10. Complete System Workflow

1. The stakeholder speaks during the live meeting while captions are enabled.
2. The Chrome Extension detects the visible caption text.
3. The extension waits until the caption becomes stable.
4. The stable statement is sent to the FastAPI /analyze-requirement endpoint.
5. The AI determines whether clarification is necessary.
6. If ambiguity is detected, the assistant displays the ambiguity and one clarification question.
7. The stakeholder response is entered into the assistant.
8. The response, original statement, and clarification question are sent to /generate-requirement.
9. The AI produces a refined requirement and classifies it as FR or NFR.
10. The refined requirement is stored in the requirements history.
11. The user can request a before-versus-after quality evaluation.
12. The evaluation is displayed and associated with the requirement.
13. The collected requirements can be exported as JSON.

11. Example of Requirement Refinement

A representative example demonstrates the central idea of the system.

Stage	Example
Original statement	The system should be fast.
Detected issue	The word 'fast' is not a measurable performance target.
Clarification question	What is the maximum acceptable processing time for one resume?
Stakeholder response	The system should process each resume within 3 seconds.
Refined requirement	The system shall process each resume within 3 seconds.
Classification	NFR — Performance

12. Requirement Quality Evaluation

The prototype uses four quality dimensions. Each dimension is scored from 1 to 5. The scoring is intended to provide a simple and understandable comparison for the academic demonstration.

Metric	Scoring interpretation
Clarity	1 = very unclear; 5 = very clear.
Completeness	1 = highly incomplete; 5 = sufficiently complete.
Testability	1 = difficult to verify objectively; 5 = easily testable.
Ambiguity	1 = highly ambiguous; 5 = very little ambiguity.

The final implementation calculates an overall before and after average from the four dimensions and presents the improvement to the user. The actual scores should be taken from the AI evaluation returned during the final demo; the report should not claim fixed scores before the live test is completed.

13. Error Handling and Limitations

The backend includes explicit handling for AI service errors. In particular, Gemini quota or rate-limit failures are returned as a 429 response with a clear message instead of appearing as an unexplained server crash. This was important during development because the Gemini free-tier quota was exhausted during testing.

The caption extraction approach relies on the visible text structure of the meeting page rather than a formal meeting platform transcript API. Therefore, changes to the meeting platform's page structure could require updates to the caption extraction logic. The prototype is intended for the assignment demonstration rather than unrestricted production deployment.

The quality scores are AI-generated and therefore should be treated as an evaluation aid rather than a formal proof of requirement quality. Human review remains important in real requirements engineering.

14. Testing Plan

Test ID	Test	Expected Result	
T1	Backend /test endpoint	Returns successful connection response.	
T2	Meet caption detection	Stable stakeholder caption is detected.	
T3	Ambiguous requirement	Ambiguity and clarification question are displayed.	
T4	Stakeholder response	Refined requirement is generated.	
T5	FR/NFR classification	Requirement receives FR/NFR type and category.	
T6	Quality evaluation	Before/after scores are displayed.	
T7	History	Requirement is retained in history.	
T8	Export	JSON file is generated successfully.	
T9	AI quota/error handling	Clear error is shown when AI service is unavailable.	

15. Results and Discussion

The prototype successfully establishes the intended requirements-engineering workflow from live meeting captions through AI analysis, clarification, refinement, classification, evaluation, history, and export. During development, caption extraction was verified against the visible meeting text and the extension successfully detected stable transcript content. The backend connection was also verified independently.

The final AI quality scores and screenshots should be inserted into this section after the Gemini quota becomes available and the complete end-to-end test is performed. This avoids presenting illustrative values as measured results.

16. Conclusion

The AI Requirement Assistant demonstrates a practical application of artificial intelligence in Software Engineering, specifically in requirements elicitation and analysis. The system does not merely convert stakeholder statements into requirements; it attempts to identify where clarification is needed, asks a targeted question, incorporates the stakeholder's response, and produces a more precise FR or NFR. The before-versus-after evaluation further provides evidence of the intended improvement in clarity, completeness, testability, and ambiguity.

The prototype therefore addresses the central assignment objective of supporting better requirements during live meetings. Future production versions could use official meeting-platform integrations and persistent requirements storage, but those capabilities are outside the necessary scope of this academic prototype.

17. References

1. Course Optional Take-Home Assignment 2 problem statement and implementation requirements.

2. Google AI for Developers documentation for Gemini API and rate limits.

3. FastAPI documentation for building Python web APIs.

4. Chrome Extensions documentation for Manifest V3 and content scripts.

Appendix A — Main API Endpoints

Endpoint	Method	Purpose
/	GET	Backend health/home message.
/test	GET	Verify extension-to-backend connectivity.
/ai-test	GET	Test Gemini connectivity.
/analyze-requirement	POST	Detect ambiguity and generate clarification question.
/generate-requirement	POST	Generate and classify the refined FR/NFR.
/evaluate-requirement	POST	Compare requirement quality before and after clarification.

Appendix B — Submission Checklist

- Source Code
- Demo Video
- Report PDF
- ZIP file containing the required submission materials
- Remove .env and API keys from the submitted source code.